SRI SIDDHARTHA ACADEMY OF HIGHER EDUCATION
(DEEMED TO BE UNIVERSITY)
Accredited A+ Grade by NAAC
SRI SIDDHARTHA INSTITUTE OF TECHNOLOGY
(A Constituent College of SSAHE, Maraluru, Kunigal Road, Tumakuru-572105)
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
(Accredited by NBA)
Email: hodcs@ssit.edu.in

LABORATORY MANUAL

For

B.E

FIRST SEMESTER

(Common to all Branches)

25CS106: C - Programming Lab

Prepared by:

Dr. Gopala T.

Associate Professor

Dept. of CS&E, SSIT

PREFACE

A good basic knowledge is a strong foundation for every aspect. C is a high-level and general-purpose programming language that is ideal for developing firmware or portable applications. Today, C is the most widely used and popular System Programming Language. Most of the state-of-the-art software's have been implemented using C.

Programming Laboratory provides an environment for learning and better understanding of the basic concepts and constructs of C programming language. The students will be able to enhance their analysing and problem-solving skills and use the same for writing programs in C. This lab manual is prepared for the lab course of C Programming Lab for the students of first year engineering.

Dr. Raviram V.
Professor & Head
Dept. of CS&E, SSIT

Dept. of CSE

Syllabus for the Academic Year – 2025 - 26

Department: Computer Science and Engineering **Semester: I**

Subject Name: C - Programming Lab

Subject Code: 25CS106 **L-T-P-S: 14-0-28-18**

Course Objectives:

Sl. No	Course Objectives
1	To introduce programming using C language.
2	To enhance analysing and problem solving skills and use the same for writing programs in C.
3	To enable effective use of branching and looping concepts in developing programs.
4	To empower skill to develop programs using derived and user defined data types.

Course Outcomes:

Course outcome	Descriptions
CO1	Elucidate the basic mathematical problems using C language.
CO2	Choose appropriate control statements of C language to solve problems.
CO3	Develop programs using branching and looping statements.
CO4	Develop solutions to problems using arrays and modular programming constructs using functions, pointers and recursions.
CO5	Explore user-defined data types like structures and unions in implementing solutions.

CO -PO MAPPING:

	PO-1	PO-2	PO-3	PO-4	PO-5	PO-6	PO-7	PO-8	PO-9	PO-10	PO-11	PSO-1	PSO-2
CO1	3	2	1	1	1	1			1		1		
CO2	3	2	3	2	1	2			1		1		
CO3	3	3	2	1	1	2			1		1		
CO4	3	3	3	2	1	1			1		1		
CO5	3	2	2								2		

Dept. of CSE

INTRODUCTION

Steps involved in Program Development:

To develop the program in high level language and translate it into machine level language following steps have to be practiced.

1. Writing and editing the program.
2. Linking the program with the required library modules.
3. Compiling the program.
4. Executing the program.

- **Software Requirement:** Linux Operating System with GCC / TURBO C in WINDOWS OS / TURBO C++ in WINDOWS OS.

STRUCTURE OF C PROGRAM:

C program is a collection of several instructions where each instruction is written as a separate statement. The C program starts with a main function followed by the opening braces which indicates the start of the function. Then follows the variable and constant declarations which are followed by the statements that include input and output statements.

Structure of C program is as shown below:

DOCUMENTATION SECTION

LINK SECTION

DEFINITION SECTION

GLOBAL DECLARATION SECTION

main() function section

{

Declaration part

Executable part

}

SUBPROGRAM SECTION

User defined functions

Dept. of CSE

LIST OF EXPERIMENTS

Cycle 1

Sl. No	Experiment Description
1	a) Develop a C program to find the area of a circle. b) Develop a C program to find the simple and compound interest.
2	a) Develop a C program to find the area of triangle for given three sides. b) Develop a C program to check whether a number is positive or negative.
3	a) Develop a C program to find a number is odd or even. b) Develop a C program to check whether a given year is leap year or not.
4	a) Develop a C program to find the biggest of given three integer numbers. b) Develop a C program to check whether a person is eligible to vote or not.
5	a) Develop a C program to find and output all the roots of a given quadratic equation for non-zero coefficients. In case of errors, your program should display suitable error messages. b) Develop a C program to perform the operations like addition, subtraction, multiplication and division only on integers. Use switch statement.

Cycle 2

Sl. No	Experiment Description
1	a) Develop a C program to display the grade of a student based on the marks. b) Develop a C program to demonstrate goto statement.
2	a) Develop a C program to check for a PERFECT number. b) Develop a C program to reverse a given integer number and check whether it is PALINDROME or not. Output the given number with suitable message.
3	a) Develop a C program to generate and display the first 'N' FIBONACCI numbers. b) Develop a C program to find the GCD and LCM of two integer numbers using Euclid's method.
4	a) Develop a C program to check a number is PRIME or not. b) Develop a C program to display and count the number of PRIME numbers in the range N1 and N2.
5	a) Develop a C program to read N integers number (zero, positive or negative) into an array and find the sum of all positive, negative and average of all input numbers. b) Develop a C program to sort N integer number in ASCENDING/DESCENDING using BUBBLE sort. Display both input array and sorted array with suitable headings.

Cycle 3

Sl. No	Experiment Description
1	a) Develop a C program to read N integer numbers into an array. Conduct a LINEAR search for a given KEY number and display success or failure in the form of a suitable messages. b) Develop a C program to read N integer numbers (in ASCENDING order) into an array. Conduct a BINARY search for a given KEY number and display success or failure in the form of a suitable messages.

Dept of CSE

2	a) Develop a C program to read two matrices A(M*N) and B(P*Q) and perform ADDITION or SUBTRACTION of A and B. Find the TRACE of the resultant matrix. b) Develop a C program to find and display the TRANSPOSE of a matrix.
3	a) Develop a C program to read two strings and CONCATENATE them (without using library functions). Output the given strings along with the concatenated string. b) Develop a C program to check a given string is PALINDROME or not.
4	a) Develop a C program to read N names and sort them in alphabetical order. b) Develop a C program to multiply to matrices using functions.
5	a) Develop a C program to demonstrate the concept of CALL BY VALUE and CALL BY REFERENCE. b) Develop a C program to demonstrate the usage of structures.

Dept. of CSE

LAB PROGRAMS

Cycle 1

1a. Develop a C program to find the area of a circle.

- **We declare variables**:
  - radius → to store user input.
  - area → to store the calculated area.
  - PI → constant value of π (3.14159).
- **We ask the user** to enter the radius.
- **We use the formula**
  - Area=π×radius^2
- **We print the area**
  - using printf() with %.2f to display two decimal places.

```
#include <stdio.h>
#define PI 3.14159
void main()
{
   float radius, area;
   // Input radius
   printf("Enter the radius of the circle: ");
   scanf("%f", &radius);
   // Calculate area
   area = PI * radius * radius;
   // Output result
   printf("Area of the circle = %.2f\n", area);
}
```

Output:

Enter the radius of the circle: 5

Area of the circle = 78.54

Dept. of CSE

b. Develop a C program to find the simple and compound interest.

Simple Interest Formula
SI=(P×R×T) / 100

Compound Interest Formula
CI= P * pow((1 + R / 100), T) - P

```
#include <stdio.h>
#include <math.h> // For pow() function
void main() {
   float principal, rate, time;
   float simpleInterest, compoundInterest;
   // Input values
   printf("Enter Principal amount: ");
   scanf("%f", &principal);
   printf("Enter Rate of Interest: ");
   scanf("%f", &rate);
   printf("Enter Time (in years): ");
   scanf("%f", &time);
   // Simple Interest calculation
   simpleInterest = (principal * rate * time) / 100;
   // Compound Interest calculation
   compoundInterest = principal * pow((1 + rate / 100), time) - principal;
   // Display results
   printf("Simple Interest = %.2f\n", simpleInterest);
   printf("Compound Interest = %.2f\n", compoundInterest);
   }
```

Output:

```
Enter Principal amount: 1000
Enter Rate of Interest: 5
Enter Time (in years): 2
Simple Interest = 100.00
Compound Interest = 102.50
```

Dept. of CSE

2 a. Develop a C program to find the area of triangle for given three sides.

Formula Used

- Semi-perimeter
  s=(a+b+c)/2
- Area
  Area=root(s×(s−a)×(s−b)×(s−c))

```
#include <stdio.h>
#include <math.h> // For sqrt() function

  void main()
{
   float a, b, c, s, area;

  // Input the sides of the triangle
   printf("Enter the three sides of the triangle: ");
   scanf("%f %f %f", &a, &b, &c);

 // Calculate semi-perimeter
   s = (a + b + c) / 2;

 // Calculate area using Heron's formula
   area = sqrt(s * (s - a) * (s - b) * (s - c));

   // Output result
   printf("Area of the triangle = %.2f\n", area);


}
```

Output:
Enter the three sides of the triangle: 3 4 5
Area of the triangle = 6.00

Dept. of CSE

b. Develop a C program to check whether a number is positive or negative.

Logic

- If **num > 0**, it's positive.
- If **num < 0**, it's negative.
- Otherwise (**num == 0**), it's zero.

```
#include <stdio.h>

void main()
{
   int num;

   // Input number
   printf("Enter a number: ");
   scanf("%d", &num);

   // Check and display result
   if (num > 0)
   {
      printf("The number is Positive.\n");
   }
   else if (num < 0)
  {
      printf("The number is Negative.\n");
   }
   else
  {
      printf("The number is Zero.\n");
   }


}
```

Output:

```
Enter a number: 5
The number is Positive.

Enter a number: -8
The number is Negative.

Enter a number: 0
The number is Zero.
```

Dept. of CSE

3 a. Develop a C program to find a number is odd or even.

- The **modulus operator** % gives the remainder when one number is divided by another.
- If a number divided by **2** gives a remainder of **0**, it's **even**.
- Otherwise, it's **odd**.

```
#include <stdio.h>

void main()

{

   int num;

   // Input number

   printf("Enter a number: ");

   scanf("%d", &num);

   // Check and display result

   if (num % 2 == 0)

         printf("The number is Even.\n");

   else

        printf("The number is Odd.\n");

   }
```

Output:

Enter a number: 10

The number is Even.

Enter a number: 7

The number is Odd.

Dept. of CSE

b. Develop a C program to check whether a given year is leap year or not.

Logic for Leap Year

A year is a leap year if:

- It is divisible by **400** OR
- It is divisible by **4** but **not** divisible by **100**.

```
#include <stdio.h>
void main()
{
   int year;
   // Input year
   printf("Enter a year: ");
   scanf("%d", &year);
   // Check leap year condition
   if ((year % 400 == 0) || ((year % 4 == 0) && (year % 100 != 0)))
            printf("%d is a Leap Year.\n", year);
   else
            printf("%d is Not a Leap Year.\n", year);
   }
```

Output:

Enter a year: 2024

2024 is a Leap Year.

Enter a year: 1900

1900 is Not a Leap Year.

Dept. of CSE

4 a. Develop a C program to find the biggest of given three integer numbers.

- Compare num1 with num2 and num3 → if both conditions true, num1 is biggest.
- Else compare num2 with num1 and num3 → if both true, num2 is biggest.
- Else num3 is biggest.

```
#include <stdio.h>

void main()

{

   int num1, num2, num3;

   // Input three numbers

   printf("Enter three integers: ");

   scanf("%d %d %d", &num1, &num2, &num3);

   // Compare to find the biggest

   if (num1 >= num2 && num1 >= num3)

         printf("%d is the biggest number.\n", num1);

   else if (num2 >= num1 && num2 >= num3)

         printf("%d is the biggest number.\n", num2);

   else

         printf("%d is the biggest number.\n", num3);

   }
```

Output:

```
Enter three integers: 25 10 15

25 is the biggest number.

Enter three integers: -5 -2 -10

-2 is the biggest number.

Enter three integers: 25 10 51

51 is the biggest number.
```

Dept. of CSE

b. Develop a C program to check whether a person is eligible to vote or not.

- In India (and most countries), the **minimum voting age is 18**.
- If age >= 18 → eligible to vote.
- Else → not eligible.

```
#include <stdio.h>
void main()
{
   int age;
   // Input age
   printf("Enter your age: ");
   scanf("%d", &age);

   // Check eligibility
   if (age >= 18)
        printf("You are eligible to vote.\n");
   else
        printf("You are not eligible to vote.\n");
   }
```

Output:

Enter your age: 20

You are eligible to vote.

Enter your age: 16

You are not eligible to vote.

Dept. of CSE

5 a. Develop a C program to find and output all the roots of a given quadratic equation for non-zero coefficients. In case of errors, your program should display suitable error messages.

Check if a*b*c = 0 → If yes, equation is not possible (error message).
Discriminant formula
D=b2−4ac
If:

- **D > 0** → Roots are **real and distinct**.
- **D = 0** → Roots are **real and equal**.
- **D < 0** → Roots are **complex roots**.

```
/* C program to find roots of a quadratic equation */

#include<stdio.h>
#include<math.h>
#include<stdlib.h>

void main()
{
        float a,b,c,disc,x1,x2,rp,ip;
       system("clear");

        // Input coefficients

        printf("enter the values of a,b and c \n");
        scanf("%f%f%f",&a,&b,&c);

       // Check for valid quadratic equation
        if(a*b*c==0)
                {
                        printf("equation is not possible \n");
                        exit(0);
                }

        // Calculate discriminant

        disc=b*b-4*a*c;

       // Check types of roots

        if(disc==0)
                {
                        printf("roots are real and equal \n");
                        x1=-b/(2*a);
                        x2=x1;
                        printf(" root1=%f  \n  root2=%f \n",x1,x2);
                }
```

Dept. of CSE

```
        else if(disc>0)
                {
                        printf("roots are real and distinct \n");
                        x1=(-b+sqrt(disc))/(2*a);
                        x2=(-b-sqrt(disc))/(2*a);
                        printf(" root1=%f \n  root2=%f \n",x1,x2);
                }
        else
                {
                        printf("roots are imaginary \n");
                        rp=-b/(2*a);
                        ip=sqrt(fabs(disc))/(2*a);
                        printf("root1=%f + i %f \n",rp,ip);
                        printf("root2=%f - i %f  \n",rp,ip);
                }
}
```

Output:

```
enter the values of a,b and c
1 2 0
equation is not possible
```

```
enter the values of a,b and c
4 4 4
The roots are real and imaginary
root1 = 0.500000 + i0.866025
root2 = 0.500000 + i0.866025
```

```
enter the values of a,b and c
 1 2 1
The roots are real and equal
root1 = -1.000000
root2 = -1.000000
```

```
enter the values of a,b and c
 1 4 3
The roots are real and distinct
root1 = -1.000000
root2 = -3.000000
```

Dept. of CSE

b. Develop a C program to perform the operations like addition, subtraction, multiplication and division only on integers. Use switch statement.

- Take two integer inputs (num1, num2).
- Display menu for operation selection.
- Use a **switch statement**:
    1. **Case 1:** Addition
    2. **Case 2:** Subtraction
    3. **Case 3:** Multiplication
    4. **Case 4:** Division (check for divide-by-zero)
    5. **Default:** Invalid input

```
#include<stdio.h>
#include<stdlib.h>
void main()
{
        int  a, b, choice;
        system (“clear”);
        printf("enter the value of a and b \n");
        scanf("%d%d",&a,&b);
        printf("1.Addition \n 2.Subtraction \n 3.Multiplication \n 4. Division \n");
        printf("Enter your choice \n");
        scanf("%d",&choice);
        switch(choice)
        {
                case 1:
                        printf("addtion=%d  \n", a+b);
                        break;

                case 2:
                        printf("subtraction=%d  \n",a-b);
                        break;

                case 3:
                        printf("multiplication=%d  \n",a*b);
                        break;

                case 4:
                        if(b==0)
                                {
                                printf("division is not possible \n");
                                exit(0);
                                }
                printf("division=%d  \n",a/b);
                break;
                default:
                        printf("invalid choice \n");
                        break;
        }
}
```

Dept. of CSE

Output:

Enter the value of a and b
4 2

1. Addition
2. Subtraction
3. Multiplication
4. Division

Enter your Choice
2
Subtraction= 2

Enter the value of a and b
2 4

1. Addition
2. Subtraction
3. Multiplication
4. Division

Enter your Choice
3
Multiplication=8

Enter the value of a and b
2 0

1. Addition
2. Subtraction
3. Multiplication
4. Division

Enter your Choice
4
division is not possible

Enter the value of a and b
4 2

1. Addition
2. Subtraction
3. Multiplication
4. Division

Enter your Choice
5
Invalid Choice

Dept. of CSE

Cycle 2

1 a. Develop a C program to display the grade of a student based on the marks.

- The program prompts the user to enter the student's marks.
- It checks whether the entered marks are within the valid range (0 to 100).
- Based on the value, it uses if-else conditions to determine the corresponding grade.
- It prints the grade to the screen.

```
#include <stdio.h>
void main()
{
   int marks;
   // Ask user for input
   printf("Enter the student's marks (0 to 100): ");
   scanf("%d", &marks);
   // Check for valid marks
   if (marks < 0 || marks > 100)
                printf("Invalid marks! Please enter a value between 0 and 100.\n");
   else
{
      // Determine grade based on marks
      if (marks >= 90)
                printf("Grade: A\n");
      else if (marks >= 80)
                printf("Grade: B\n");
      else if (marks >= 70)
                printf("Grade: C\n");
      else if (marks >= 60)
                printf("Grade: D\n");
      else if (marks >= 50)
                printf("Grade: E\n");
      else
                printf("Grade: F (Fail)\n");
   }
}
```

Output:

```
Enter the student's marks (0 to 100): 85
Grade: B

Enter the student's marks (0 to 100): 45
Grade: F (Fail)

Enter the student's marks (0 to 100): 102
Invalid marks! Please enter a value between 0 and 100.
```

Dept. of CSE

b. Develop a C program to demonstrate goto statement.

- The user is prompted to enter a number.
- If the number is **negative**, the program jumps to the label error: using goto, and prints an error message.
- If the number is **positive**, it prints the number and jumps to the end: label, skipping the error message.
- The program always prints "Program ended." at the end.

```
#include <stdio.h>
void main()
{
   int num;

   printf("Enter a number (positive): ");
   scanf("%d", &num);

   if (num < 0)
        goto error;  // jump to error label if input is negative

   printf("You entered a positive number: %d\n", num);
   goto end; // jump to end to skip the error message

error:
   printf("Error: You entered a negative number!\n");

end:
   printf("Program ended.\n");

  }
```

Output:

```
Enter a number (positive): 10
You entered a positive number: 10
Program ended.

Enter a number (positive): -5
Error: You entered a negative number!
Program ended.
```

Dept. of CSE

2 a. Develop a C program to check for a PERFECT number.

- The program asks the user to enter a positive number.
- It loops from 1 to num - 1, checking if each number divides num evenly.
- If so, it adds that divisor to the sum.
- After the loop, if sum == num, it's a perfect number.

```
#include <stdio.h>
void main()
{
   int num, i, sum = 0;
   // Get input from user
   printf("Enter a positive integer: ");
   scanf("%d", &num);

   // Check for valid input
   if (num <= 0) {
      printf("Please enter a positive integer.\n");
      return 0;
   }

   // Find sum of proper divisors
   for (i = 1; i < num; i++)
   {
         if (num % i == 0)
               sum += i;

   }

   // Check if number is perfect
   if (sum == num)
      printf("%d is a PERFECT number.\n", num);
   else
      printf("%d is NOT a perfect number.\n", num);
}
```

Output:

```
Enter a positive integer: 6
6 is a PERFECT number.

Enter a positive integer: 10
10 is NOT a perfect number.
```

Dept. of CSE

b. Develop a C program to reverse a given integer number and check whether it is PALINDROME or not. Output the given number with suitable message.

- **Input** a number from the user.
- Store the **original number** for comparison later.
- Use a **while loop** to extract digits and build the **reversed number**.
- After reversal, compare it with the original.
- Print the reversed number and whether it's a palindrome.

```
#include<stdio.h>
void main()
{
int n,temp,n,rev=0,digit;
printf("enter the value of n \n");
    scanf("%d",&n);

temp=n;

while(temp!=0)
        {
                digit=temp%10;
                rev=rev*10+digit;
                temp=temp/10;
        }

printf("\n reversed number=%d",rev);

if(n==rev)
        printf("%d is a palindrome \n",n);
else
        printf("%d is not a palindrome \n",n);
}
```

Output:

```
enter the value of n
1234
reversed number=4321

1234 is not a palindrome


enter the value of n
1221
reversed number=1221

1221 is a palindrome
```

Dept. of CSE

3 a. Develop a C program to generate and display the first 'N' FIBONACCI numbers.

The user inputs how many Fibonacci terms (n) to generate.

- The program initializes the first two terms: 0 and 1.
- A loop runs from 1 to n:
- Each number is printed in order.

```
#include <stdio.h>
void main()
{
int  i, n,f1=0,f2=1,f3;
          printf("Enter the value of n: ");
          scanf("%d", &n);

for(i<0;i<n;i++)
   {
         printf("%d\t",f1);
         f3=f1+f2;
         f1=f2;
         f2=f3;
   }
}
```

Output:

Enter a positive integer: 6

Fibonacci Series: 0 1 1 2 3 5

Dept. of CSE

b. Develop a C program to find the GCD and LCM of two integer numbers using Euclid's method.

Input: Two positive integers.

- **GCD** is calculated using a while loop based on Euclid's algorithm.
- **LCM** is calculated using the formula: LCM= (num1×num2)/ GCD

```
#include<stdio.h>
#include<stdlib.h>
 void main()
{
int n1,n2,nr,dr,rem,gcd,lcm;
system("clear");
printf("\n enter the value of n1 and n2\n");
        scanf("%d%d",&n1,&n2);
if(n1>n2)
    {
        nr=n1;
        dr=n2;
    }
else
  {
        nr=n2;
        dr=n1;
  }
rem=nr%dr;

while(rem!=0)
  {
        nr=dr;
        dr=rem;
        rem=nr%dr;
  }
gcd=dr;
lcm=n1*n2/gcd;
printf("\n GCD(%d,%d)=%d\n",n1,n2,gcd);
printf("\n LCM(%d,%d)=%d\n",n1,n2,lcm);
}
```

Output:

```
Enter the value of n1 and n2
2 4
GCD(2,4)=2
LCM(2,4)=4

Enter the value of n1 and n2
4 2
GCD(4,2)=2
LCM(4,2)=4
```

Dept. of CSE

4 a. Develop a C program to check a number is PRIME or not.

- The user inputs a positive integer.
- The program handles the special case: 0, 1, and negative numbers are not prime.
- A loop checks if the number is divisible by any number from 2 to num/2.
- If any divisor is found, the number is **not prime**.
- Otherwise, it is **prime**.

```
#include <stdio.h>
#include<stdlib.h>
void main()
{
   int num, i;
   // Input from user
   printf("Enter a positive integer: ");
   scanf("%d", &num);
   // Handle special cases
   if (num <= 1)
  {
      printf("%d is NOT a prime number.\n", num);
      return 0;
   }

   // Check divisibility from 2 to sqrt(num) (simplified here up to num/2)
   for (i = 2; i <= num / 2; i++)
  {
      if (num % i == 0)
        {
           printf("%d is NOT a prime number.\n", num);
           exit(0);
        }
   }

  printf("%d is a PRIME number.\n", num);

 }
```

Output:

Enter a positive integer: 7
7 is a PRIME number.

Enter a positive integer: 10
10 is NOT a prime number.

Dept. of CSE

b. Develop a C program to display and count the number of PRIME numbers in the range N1 and N2.

```
#include<stdio.h>
#include<stdlib.h>
void main()
{
        int  n1,n2,i,j,flag,count=0;
        system(‘clear”);

        printf("enter the value of N1 and N2\n");
        scanf("%d%d",&n1,&n2);

      printf(“Prime Numbers between %d and %d are\n”,n1,n2);

      for (i=n1;i<=n2;i++)
       {
            flag=0;
         for(j=2;j<=i/2;j++)
         {
              if(i%j==0)
             {
                flag=1;
                break;
              }
         }
        if(flag==0)
         {
              count++;
              printf(“%d\t”,i);
          }
      }
      printf(“Number of prime numbers=%d\n”,count);
}
```

Output:

```
enter the value of N1 and N2
 1   10
Prime numbers between 1 and 10 are
1        2        3        5        7
No. of prime numbers= 5
```

Dept. of CSE

5 a. Develop a C program to read N integers number (zero, positive or negative) into an array and find the sum of all positive, negative and average of all input numbers.

Loop runs **n** times:

- Each number is read and:
  - If it's positive → added to psum.
  - If it's negative → added to nsum.
- Average is calculated by using this formula (psum+nsum)/n.

```
#include<stdio.h>
void main()
{       int a[50],n,i;
        float psum=0,nsum=0,avg;
        printf("\n enter the array size\n");
        scanf("%d",&n);
        printf("\n enter the array elements\n");
        for(i=0;i<n;i++)
            scanf("%d",&a[i]);

        for(i=0;i<n;i++)
        {
            if(a[i]>0)
                    psum=psum+a[i];
             else
                    nsum=nsum+a[i];
        }
        avg=(psum+nsum)/n;
        printf("\n positive sum=%f\n",psum);
        printf("\n negative sum=%f\n",nsum);
        printf("\n average=%f\n",avg);
}
```

Dept. of CSE

Output:

enter the array size

5

enter the array elements

2 -15 6 0 -3

positive sum = 8.000000

negative sum = -18.000000

average =-2.000000

Dept. of CSE

b. Develop a C program to sort N integer number in ASCENDING/DESCENDING using BUBBLE sort. Display both input array and sorted array with suitable headings.

- **Input:** The user inputs the number of integers and then the integers themselves.
- All arrays original and sorted (ascending/descending) are displayed with headings.

```
#include<stdio.h>
void main()
{
        int a[10],i,j,n,temp;
        system("clear");
        printf("enter the size of the array \n");
           scanf("%d",&n);
        printf("enter the elements of the array \n");
        for(i=0;i<n;i++)
            scanf("%d",&a[i]);
        printf(" Array elements - Before sorting \n");
        for(i=0;i<n;i++)
            printf("%d\t",a[i]);

        /* Bubble Sort takes place below */
        for(i=0;i<n-1;i++)
                for(j=0;j<n-i-1;j++)
                        if(a[j]>a[j+1])
                                {
                                temp=a[j];
                                a[j]=a[j+1];
                                a[j+1]=temp;
                                }
        printf("\n Array elements - After sorting \n");
        for(i=0;i<n;i++)
                printf("%d\t",a[i]);
}
```

Output:

```
enter the size of the array
5
enter the elements of the array
10
40
30
20
50
Array elements - Before sorting
10      40      30      20      50

Array elements - After sorting

10      20      30      40      50
```

Dept. of CSE

Cycle 3

1 a. Develop a C program to read N integer numbers into an array. Conduct a LINEAR search for a given KEY number and display success or failure in the form of a suitable messages.

Here's a simple **C program** that:

- Reads **N integer numbers** into an array.
- Accepts a **key** to search for.
- Performs a **linear search**, and displays a **success or failure message**.

```
#include<stdio.h>
#include<stdlib.h>
 main()
{
   int a[20],i,key,n;
   printf("enter the size of the array \n");
        scanf("%d",&n);
   printf("Enter array elements:\n");
      for(i=0;i<n;i++)
        scanf("%d",&a[i]);
   printf("\nEnter element to search searched: \n");
      scanf("%d",&key);
    for(i=0;i<n;i++)
        if(a[i]==key)
                {
                   printf("Element found at index %d \n",i+1);
                   exit(0);
                }
    printf("Element not found");
 }
```

Output:

enter the size of the array

5

Enter array elements:

21 22 34 12 66

Enter element to searched

34

Element found at index 3

Dept. of CSE

b. Develop a C program to read N integer numbers (in ASCENDING order) into an array. Conduct a BINARY search for a given KEY number and display success or failure in the form of a suitable messages.

Here is a **simple C program** that:

- Reads N integers in **ascending order** into an array
- Accepts a **key** to search for
- Performs a **binary search** and displays whether the key was **found or not**

```
#include<stdio.h>
#include<stdlib.h>
void main()
{
        int a[20],n,i,low,high,mid,key;
        system("clear");

        printf("enter the size of the array \n");
           scanf("%d",&n);

        printf("enter the array elements \n");
           for(i=0;i<n;i++)
               scanf("%d",&a[i]);

        printf("enter the key element to be searched \n");
             scanf("%d",&key);

        low=0;
        high=n-1;

        while(low<=high)
                {
                  mid=(low+high)/2;
                        if(key==a[mid])
                                {
                                printf("key element is found at position %d \n",mid+1);
                                exit(0);
                                }
                        if(key>a[mid])
                                low=mid+1;
                        else
                                high=mid-1;
                }

        printf("key element is not found\n");
}
```

Dept. of CSE

Output:

```
enter the size of the array
5
enter the array elements
1
2
3
4
5
enter the key element to be searched
4
key element is found at position 4
```

```
enter the size of the array
5

enter the array elements
1
2
3
4
5
enter the key element to be searched
7
key element is not found
```

Dept. of CSE

2 a. Develop a C program to read two matrices A(M*N) and B(P*Q) and perform ADDITION or SUBTRACTION of A and B. Find the TRACE of the resultant matrix.

Here's a **simple C program** that:

- Reads two matrices A and B
- Performs **Addition or Subtraction** (only if their sizes are equal)
- Finds the **Trace (***sum of diagonal elements***)** of the Resultant Matrix

```
#include<stdio.h>
void main()
{
        int a[10][10],b[10][10],c[10][10],m,n,i,j,trace=0,ch;
        printf("\n enter the number of rows and columns\n");
        scanf("%d%d",&m,&n);

        printf("\n enter the array elements of matrix A\n");
        for(i=0;i<m;i++)
           for(j=0;j<n;j++)
             scanf("%d",&a[i][j]);
        printf("\n enter the array elements of matrix B\n");
        for(i=0;i<m;i++)
           for(j=0;j<n;j++)
              scanf("%d",&b[i][j]);
                 printf("\n enter 1.Addition and \n 2.Subtraction\n");
        scanf("%d",&ch);

        switch(ch)
        {
                case 1:for(i=0;i<m;i++)
                          for(j=0;j<n;j++)
                          c[i][j]=a[i][j]+b[i][j];
                          break;
                case 2:for(i=0;i<m;i++)
                          for(j=0;j<n;j++)
                          c[i][j]=a[i][j]-b[i][j];
                          break;
                default: printf("\n invalid choice");
        }

printf("\n Matrix A is\n");
for(i=0;i<m;i++)

{

        for(j=0;j<n;j++)
        printf("%d\t",a[i][j]);
        printf("\n");
}
```

Dept. of CSE

```
printf("\n Matrix B is\n");
for(i=0;i<m;i++)
{
        for(j=0;j<n;j++)
        printf("%d\t",b[i][j]);
        printf("\n");
}

if(ch==1)
     printf("\n resultant matrix for addition is\n");
else
      printf("\n resultant matrix for subtraction is\n");

for(i=0;i<m;i++)
{
        for(j=0;j<n;j++)
        printf("%d\t",c[i][j]);
        printf("\n");
}

if(m==n)
{
     for(i=0;i<n;i++)
           trace=trace+c[i][i];
     printf("\n trace=%d",trace);
}
else
printf("trace is not possible");
}
```

Output:

```
enter the number of rows and columns
2 3
enter the array elements of matrix A
1
1
1
1
1
1
enter the array elements of matrix B
1
1
1
1
1
```

Dept. of CSE

```
1
Enter 1 for addition and
    2 for subtraction
1
Matrix A is
1   1   1
1   1   1
Matrix B is
1   1   1
1   1   1
resultant matrix for addition is
2   2   2
2   2   2
trace is not possible
```

```
enter the number of rows and columns
2 2
enter the array elements of matrix A
1
1
1
1
enter the array elements of matrix B
1
1
1
1
Enter 1 for addition and
    2 for subtraction
1
Matrix A is
1   1
1   1
Matrix B is
1   1
1   1
resultant matrix for addition is
2   2
2   2
trace = 4
```

Dept. of CSE

b. Develop a C program to find and display the TRANSPOSE of a matrix.

Here's a **simple C program** that:

- Transpose of a matrix is obtained by interchanging rows and columns.
- If matrix A is of size m × n, its transpose A^t is of size n × m.
- This program works for any m x n matrix, not just square matrices.

```
#include <stdio.h>
void main()
{
   int A[10][10], T[10][10],i, j, rows, cols;
   // Step 1: Input number of rows and columns
   printf("Enter number of rows and columns: ");
      scanf("%d %d", &rows, &cols);
   // Step 2: Input matrix elements
   printf("Enter elements of the matrix:\n");
   for(i = 0; i < rows; i++)
         for(j = 0; j < cols; j++)
            scanf("%d", &A[i][j]);
   // Step 3: Find Transpose (swap rows and columns)
   for(i = 0; i < rows; i++)
      for(j = 0; j < cols; j++)
          T[j][i] = A[i][j];
    // Step 4: Display original matrix
   printf("Original Matrix:\n");
   for(i = 0; i < rows; i++)
    {
      for(j = 0; j < cols; j++)
           printf("%d\t ", A[i][j]);
      printf("\n");
   }
```

Dept. of CSE

```
    // Step 5: Display transposed matrix
    printf("Transpose of the Matrix:\n");
    for(i = 0; i < cols; i++)
    {
       for(j = 0; j < rows; j++)
            printf("%d\t ", T[i][j]);
       printf("\n");
    }
}
```

Output:

Enter number of rows and columns: 2 3

Enter elements of the matrix:

1

2

3

4

5

6

Original Matrix:

1 2 3

4 5 6

Transpose of the Matrix:

1 4

2 5

3 6

Dept. of CSE

3 a. Develop a C program to read two strings and CONCATENATE them (without using Library functions). Output the given strings along with the concatenated string.

```
#include<stdio.h>
#include<string.h>
void main()
{
  char str1[10],str2[10];
  int i=0,j=0,len=0;
  system("clear");
  printf("enter the first string\n");
      gets(str1); // Reads string without spaces

  printf("enter the second string\n");
      gets(str2); // Reads string without spaces

for(i=0;str1[i]!='\0';i++); //moving cursor to end of the first string

// Append str2 to str1

while(str2[j]!='\0')

{

       str1[i]=str2[j];

       i++;

       j++;

}

// Null-terminate the concatenated string

          str1[i] = '\0';

  printf("the concatenated string %s \n",str1);
}
```

Output:

```
enter the first string
siddhartha

enter the second string
university

the concatenated string=siddharthauniversity
```

Dept. of CSE

b. Develop a C program to check a given string is PALINDROME or not.

```
#include<stdio.h>
#include<conio.h>
void main()
{
char str[60];
int i,j,count=0;k
clrscr();
printf("\n enter any string to check for palindrome or not \n");
        gets(str)
for(k=0;str[k]!='\0';k++0)
        count++;
i=0, j=count-1;
while(i<j)
  {
        if(str[i]!=str[j])
                {
                    printf("\n %s is not a palindrome \n",str);
                    getch();
                    exit(0);
                 }
        i++;
        j--;
  }
 printf("\n %s is a palindrome \n", str);
}
```

Output:

```
enter any string to check for palindrome or not
madam
madam is a palindrom

enter any string to check for palindrome or not
SSIT
SSIT is not a palindrome
```

Dept. of CSE

4 a. Develop a C program to read N names and sort them in alphabetical order.

```
#include <stdio.h>
#include <string.h>
void main()
{
   char names[50][50], temp[50];
   int i, j, n;
   // Read number of names
   printf("Enter number of names: ");
         scanf("%d", &n);
   // Read names
   printf("Enter %d names:\n", n);
   for (i = 0; i < n; i++)
       scanf("%s", &names[i]);

// Sort names using bubble sort
   for (i = 0; i < n-1; i++)
 {
      for (j =0; j < n-i-1; j++)
          {
             if (strcmp(names[j], names[j+1]) > 0)
                 {
                    // Swap names[i] and names[j]
                        strcpy(temp, names[j]);
                        strcpy(names[j], names[j+1]);
                        strcpy(names[j+1], temp);
                 }
          }
   }
```

Dept. of CSE

```
 // Print sorted names
   printf("\n Names in alphabetical order:\n");
   for (i = 0; i < n; i++)
        printf("%s\n", names[i]);
}
```

Output:

Enter number of names: 6

Enter 6 names:

mivan

saanvi

hema

gopala

raviram

tanmay

Names in alphabetical order:

gopala

hema

mivan

raviram

saanvi

tanmay

Dept. of CSE

b. Develop a C program to multiply to matrices using functions.

Here's a **simple C program** to multiply two matrices using **functions**.

This program includes:

- A function to **read** a matrix
- A function to **display** a matrix
- A function to **multiply** two matrices

```
#include<stdio.h>
#include<stdlib.h>

void readmatrix(int x[10][10],int m,int n);
void writematrix(int x[10][10],int m,int n);
void productmatrix(int a[10][10],int b[10][10],int c[10][10],int m,int n,int q);

void main()
{
        int m,n,p,q,a[10][10],b[10][10],c[10][10];
        system("clear");
        printf("enter the value of m and n\n");
            scanf("%d%d",&m,&n);

        printf("enter the value of p and q\n");
            scanf("%d%d",&p,&q);

        if(n!=p)
        {
             printf("multiplication not possible\n");
             exit(0);
             getch();
        }
        printf("enter the elements for the matrix A\n");
             readmatrix(a,m,n);

        printf("enter the elements for the matrix B \n");
             readmatrix(b,p,q);

        printf("matrix A is\n");
              writematrix(a,m,n);

        printf("matrix B is\n");
             writematrix(b,p,q);
         productmatrix(a,b,c,m,n,q);
        printf("resultant matrix after multiplication is \n");
            writematrix(c,m,q);
        }
```

Dept. of CSE

```
void readmatrix(int x[10][10],int r,int c)
{
int i,j;
for(i=0;i<r;i++)
      for(j=0;j<c;j++)
          scanf("%d",&x[i][j]);
}
void writematrix(int x[10][10],int r,int c)
{
int i,j;
for(i=0;i<r;i++)
{
        for(j=0;j<c;j++)

                    printf("%d\t",x[i][j]);
      printf("\n");
}
void productmatrix(int a[10][10],int b[10][10],int c[10][10],int m,int p,int q)
{
int i,j,k;
for(i=0;i<m;i++)
     {
          for(j=0;j<q;j++)
                {
                        c[i][j]=0;
                              for(k=0;k<n;k++)
                                  c[i][j]=c[i][j]+a[i][k]*b[k][j];
                 }
     }
 }
```

Output:

```
enter the value of m and n
2
2
enter the value of p and q
2
2
enter the elements for the matrix A
1 2 3 4
enter the elements for the matrix B
5 6 7 8
matrix A is
   1   2
   3   4
matrix B is
   5   6
   7   8
resultant matrix after multiplication is
    19   22

    43   50
```

Dept. of CSE

5 a. Develop a C program to demonstrate the concept of CALL BY VALUE and CALL BY REFERENCE.

Concept Summary:

- **Call by Value:** A copy of the variable is passed. Changes **do not affect** the original variable.
- **Call by Reference:** The **address** of the variable is passed. Changes **affect** the original variable.

CALL BY VALUE PROGRAM:

```
#include <stdio.h>
#include<conio.h>
void swap(int , int ) ;

void main()
{
 int a = 40, b = 50;
 clrscr();
 printf("Before swap, a = %d and b = %d\n", a, b);
    swap(a, b);
 printf("After swap, a = %d and b = %d\n", a, b);
}


void swap(int x, int y)
{
 int temp;
      temp = x;
      x = y;
      y = temp;
}
```

Output:

Before swap, a=40 and b=50
After swap, a=40 and b=50

Dept. of CSE

CALL BY REFERENCE PROGRAM:

```
#include <stdio.h>
#include<conio.h>
void swap(int *, int *);

void main ()
 {
   int a = 100, b = 200;
   clrscr();
   printf("Before swap, value of a : %d\n", a );
   printf("Before swap, value of b : %d\n", b );

   swap(&a, &b);

   printf("After swap, value of a : %d\n", a );
   printf("After swap, value of b : %d\n", b );
}


void swap(int *x, int *y)
{
    int temp;
          temp = *x; /* save the value of x */
          *x = *y;    /* put y into x */
          *y = temp; /* put temp into y */

}
```

Output:

```
Before swap, value of a: 100
Before swap, value of b: 200

After swap, value of a: 200
After swap, value of b: 100
```

Dept. of CSE

b. Develop a C program to demonstrate the usage of structures.

```
#include <stdio.h>
// Create a structure
struct Student
{
   int roll;
   char name[20];
   float marks;
};

void main()
{
   struct Student s;
   // Read data
   printf("Enter roll number: ");
         scanf("%d", &s.roll);
   printf("Enter name: ");
         scanf("%s", s.name);
   printf("Enter marks: ");
         scanf("%f", &s.marks);

   // Display data
   printf("\nStudent Details:\n");
   printf("Roll Number: %d\n", s.roll);
   printf("Name: %s\n", s.name);
   printf("Marks: %.2f\n", s.marks);
   }
```

Dept. of CSE

Output:

Enter roll number: 1

Enter name: Anu

Enter marks: 89.5

Student Details:

Roll Number: 1

Name: Anu

Marks: 89.50

Dept. of CSE